# Retrieval-augmented generation (RAG) in Azure AI Search

> **Note**
>
> Azure AI Search is available through the Azure portal, REST APIs, and Azure SDKs. It also underpins Foundry IQ, the managed knowledge layer that transforms enterprise content into reusable, permission-aware knowledge bases for agents in the Microsoft Foundry portal.

Retrieval-augmented generation (RAG) is a pattern that extends LLM capabilities by grounding responses in your proprietary content. While conceptually simple, RAG implementations face significant challenges.

## The challenges of RAG

Expand table

| Challenge | Description |
|---|---|
| **Query understanding** | Modern users ask complex, conversational, or vague questions with assumed context. Traditional keyword search fails when queries don't match document terminology. For RAG, an information retrieval system must understand intent, not just match words. |
| **Multi-source data access** | Enterprise content spans SharePoint, databases, blob storage, and other platforms. Creating a unified search corpus without disrupting data operations is essential. |
| **Token constraints** | LLMs accept limited token inputs. Your retrieval system must return highly relevant, concise results - not exhaustive document dumps. |
| **Response time expectations** | Users expect AI-powered answers in seconds, not minutes. The retrieval system must balance thoroughness with speed. |
| **Security and governance** | Opening private content to LLMs requires granular access control. Users and agents must only retrieve authorized content. |

# How Azure AI Search meets RAG challenges

Azure AI Search provides two approaches designed specifically for these RAG challenges:

- **Agentic retrieval**: A complete RAG pipeline with LLM-based query planning (preview), multi-source access, and structured responses optimized for agent consumption.
- **Classic RAG pattern**: The proven approach using hybrid search and semantic ranking, ideal for simpler requirements or when generally available (GA) features are required.

The following sections explain how each approach solves specific RAG challenges.

## Solving query understanding challenges

**The problem:** Users ask "What's our PTO policy for remote workers hired after 2023?" but documents say "time off," "telecommute," and "recent hires."

**Agentic retrieval solution:**

- LLM analyzes the question and generates multiple targeted subqueries.
- Decomposes complex questions into focused searches.
- Uses conversation history to understand context.
- Parallel execution across knowledge sources.

**Classic RAG solution:**

- Hybrid queries combine keyword and vector search for better recall.
- Semantic ranking re-scores results based on meaning, not just keywords.
- Vector similarity search matches concepts, not exact terms.

Learn more about query planning (preview).

## Solving multisource data challenges

**The problem:** HR policies in SharePoint, benefits in databases, company news on web pages - creating copies disrupts governance and routine data operations.

**Agentic retrieval solution:**

- Knowledge bases unify multiple knowledge sources.

- Direct query against remote SharePoint and Bing (no indexing needed) to supplement index content.
- Retrieval instructions guide the LLM to appropriate data sources.
- Automatic indexing pipeline generation for Azure Blob, OneLake, ingested SharePoint content, ingested other external content.
- Single query interface and query plan across all sources.

**Classic RAG solution:**

- Indexers pull from more than 10 Azure data sources.
- Skills pipeline for chunking, vectorization, image verbalization, and analysis.
- Incremental indexing keeps content fresh.
- You control what's indexed and how.

Learn more about knowledge sources.

## Solving token constraint challenges

**The problem:** GPT-4 accepts about 128k tokens, but you have 10,000 pages of documentation. Sending everything wastes tokens and degrades quality.

**Agentic retrieval solution:**

- Returns a structured response with only the most relevant chunks
- Built-in citation tracking shows provenance
- Query activity log explains what was searched
- Optional answer synthesis reduces token usage further

**Classic RAG solution:**

- Semantic ranking identifies the top 50 most relevant results
- Configurable result limits (top-k for vectors, top-n for text) and minimum thresholds
- Scoring profiles boost critical content
- Select statement controls which fields are returned

Learn more about relevance tuning.

## Solving response time challenges

**The problem:** Users expect answers in 3-5 seconds, but you're querying multiple sources with complex processing.

**Agentic retrieval solution:**

- Parallel subquery execution (not sequential)
- Adjustable reasoning effort (minimal/low/medium)
- Pre-built semantic ranking (no extra orchestration)

**Classic RAG solution:**

- Millisecond query response times
- Single-shot queries reduce complexity
- You control timeout and retry logic
- Simpler architecture with fewer failure points

## Solving security challenges

**The problem:** Finance data should only be accessible to finance team, even when an executive asks the chatbot.

**Agentic retrieval solution:**

- Knowledge source-level access control
- Inherits SharePoint permissions for queries against remote SharePoint
- Inherits Microsoft Entra ID permission metadata for indexed content from Azure Storage
- Filter-based security at query time for other data sources
- Network isolation via private endpoints

**Classic RAG solution:**

- Document-level security trimming
- Inherits Microsoft Entra ID permission metadata for indexed content from Azure Storage
- Filter-based security at query time for other data sources
- Network isolation via private endpoints

Learn more about security.

## Modern RAG with agentic retrieval

Azure AI Search is a proven solution for RAG workloads. It now provides agentic retrieval, a specialized pipeline designed specifically for RAG patterns. This approach uses LLMs to intelligently break down complex user queries into focused subqueries, executes them in parallel, and returns structured responses optimized for chat completion models.

Agentic retrieval represents the evolution from traditional single-query RAG patterns to multi-query intelligent retrieval, providing:

- Context-aware query planning using conversation history
- Parallel execution of multiple focused subqueries
- Structured responses with grounding data, citations, and execution metadata
- Built-in semantic ranking for optimal relevance
- Optional answer synthesis that uses an LLM-formulated answer in the query response

You need new objects for this pipeline: one or more knowledge sources (each one points to searchable content, such as a search index or a remote SharePoint site), a knowledge base (the queryable object that unifies those sources), and the retrieve action that you call from application code, such as a tool that works with your AI agent.

For new RAG implementations, start with agentic retrieval. For existing solutions, consider migrating to take advantage of improved accuracy and context understanding.

## Classic RAG pattern for Azure AI Search

Classic RAG uses the original query execution architecture where your application sends a single query to Azure AI Search and orchestrates the handoff to an LLM separately. Your deployed LLM formulates an answer using the flattened result set from the query. This approach is simpler with fewer components, and faster because there's no LLM involvement in query planning.

For detailed information about implementing classic RAG, see the azure-search-classic-rag repository.

# Content preparation for RAG

RAG quality depends on how you prepare content for retrieval. Azure AI Search supports:

Expand table

| Content challenge | How Azure AI Search helps |
| --- | --- |
| **Large documents** | Automatic chunking (built-in or via skills) |
| **Multiple languages** | More than 50 language analyzers for text, multilingual vectors |
| **Images and PDFs** | OCR, image analysis, image verbalization, document extraction skills |
| **Need for similarity search** | Integrated vectorization (Azure OpenAI, Azure Vision in Foundry Tools, custom) |
| **Terminology mismatches** | Synonym maps, semantic ranking |

**For agentic retrieval:** Use knowledge sources that auto-generate chunking and vectorization pipelines.

**For classic RAG:** Use indexers and skillsets to build custom pipelines, or push pre-processed content via the push API.

## Maximize relevance and recall

How do you provide the best grounding data for LLM answer formulation? It's a combination of having appropriate content, smart queries, and query logic that can identify the best chunks for answering a question.

During indexing, use chunking to subdivide large documents so that portions can be matched on independently. Include a vectorization step to create embeddings used for vector queries.

On the query side, to ensure the most relevant results for your RAG implementation:

- Use hybrid queries that combine keyword (nonvector) and vector search for maximum recall. In a hybrid query, if you double down on the same input, a text string and its vector equivalent generate parallel queries for keywords and similarity search, returning the most relevant matches from each query type in a unified result set.
- Use semantic ranking, built into agentic retrieval, optional for classic RAG.
- Apply scoring profiles to boost specific fields or criteria.
- Fine-tune with vector query parameters for vector weighting and minimum thresholds.

For more information, see hybrid search and semantic ranking.

# Choose between agentic retrieval and classic RAG

**Use agentic retrieval when:**

- Your client is an agent or chatbot.
- You need the highest possible relevance and accuracy.
- Your queries are complex or conversational.
- You want structured responses with citations and query details.
- You're building new RAG implementations.

**Use classic RAG when:**

- You need generally available (GA) features only.
- Simplicity and speed are priorities over advanced relevance.
- You have existing orchestration code you want to preserve.
- You need fine-grained control over the query pipeline.

A RAG solution that includes agents and Azure AI Search can benefit from Foundry IQ, as an agent's single endpoint to a knowledge layer that provides grounding data. Foundry IQ uses agentic retrieval.

Learn more about classic search, agentic retrieval, and how they compare.

# How to get started

There are many ways to get started, including code-first solutions and demos.

Videos

- Foundry IQ: The future of RAG with knowledge retrieval and Azure AI Search
- Build agents with knowledge, agentic RAG, and Azure AI Search
- (Classic RAG) Vector search and state of the art retrieval for Generative AI apps

**Note:** The author created this article with assistance from AI. Learn more

Last updated on 08/04/2026